



BUILD A WORLD MODEL FROM SCRATCH · LECTURE 4

Dreams

That Last

Last time our world model could see one step ahead — and then its dream fell apart. Today we rebuild it so the dream *holds*. And we do it on a real robot arm, with real recorded data.

BELIEF

PRIOR & POSTERIOR

MEMORY

RSSM

Vizuara AI Labs · Build a World Model from Scratch

Two seconds of a robot, imagined

no camera input for 1 steps — only the actions

the real robot



the world model's dream



dream step 01 / 60

What you are seeing

Left: a real SO-101 robot arm doing a pick-and-place, recorded on video. Right: **a neural network imagining that same motion**. After the first few frames the camera is switched off entirely — the model is told only which joint commands were sent, and it paints everything else itself, sixty steps into the future.

Why this is hard

This is the exact test our Lecture 3 model failed. Given its own predictions to eat, it drifted within a handful of steps. Today's model runs for **60 steps on a real robot scene** and stays locked to reality. The reason it can is one idea — and this lecture is that idea.

Lecture 3 ended on an honest failure

Our first world model predicted the *next* frame almost perfectly. But run in a closed loop — its own output becoming its next input — it degraded fast.

Errors compound

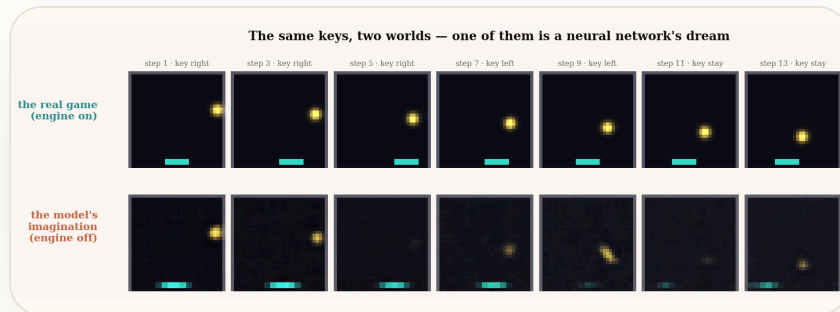
A tiny mistake at step 5 is eaten at step 6, amplified at step 7. Nothing in the machinery pushes the model back toward reality.

The encoder never knew its job

We trained the encoder *first*, alone, to redraw frames. It was never told its codes would later be chained through a predictor. So it built a code space that decodes beautifully and predicts badly.

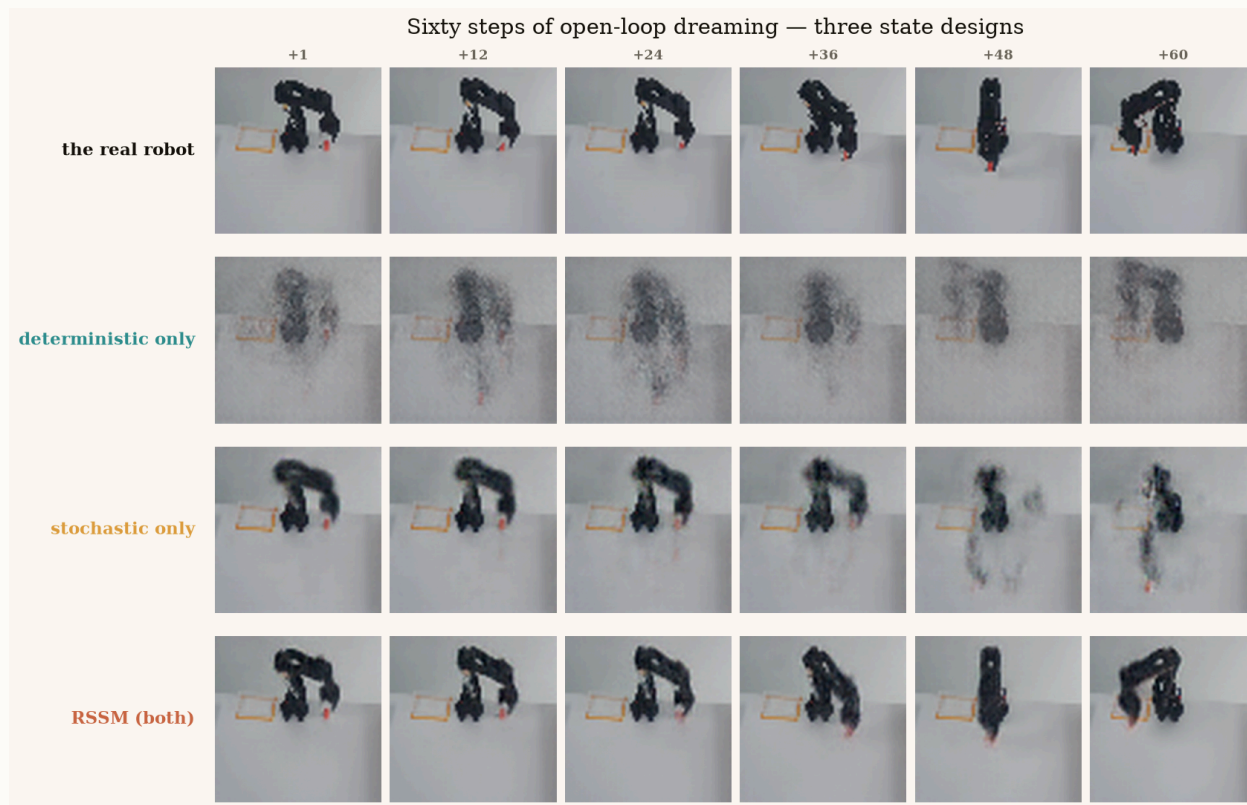
We patched it by hand

Remember our "smoothness" fix — forcing consecutive frames to get nearby codes? That was us guessing at a principle. Today we meet the real one.



Lecture 3, closed loop: the paddle obeys, but the imagined ball dissolves.

Where this lecture is going: the same robot, dreamed by three designs



row 1 the real robot · rows 2 and 3 the two designs most people would try first, both of which fail · row 4 the design we build today — we will earn every

row

One paper, one robot, three questions

The paper

PlaNet — *Learning Latent Dynamics for Planning from Pixels* (Hafner et al., 2019). It introduces the **RSSM**: the state design behind every Dreamer agent since.

The robot

SO-101 pick-and-place — 50 tele-operated episodes, 11,939 frames, openly available. Real camera, real joint angles, real contact. Everything here is trained and measured on it.

1 WHAT does the model track?

Not a state — a *belief*. A distribution over states: wide when the model is guessing, narrow when it knows.

2 HOW is the machine built?

Two designs anyone would try first, both tested on our robot and both broken — then the one that works. By the end of this part you will know exactly what an RSSM is.

3 HOW do we train it?

Only once the machine is clear do we ask how to teach it. Two losses, motivated one at a time, ending with one real training step and its real numbers.

Explanation from Rajat

— WHITEBOARD —



PART 1 OF 3

The Belief

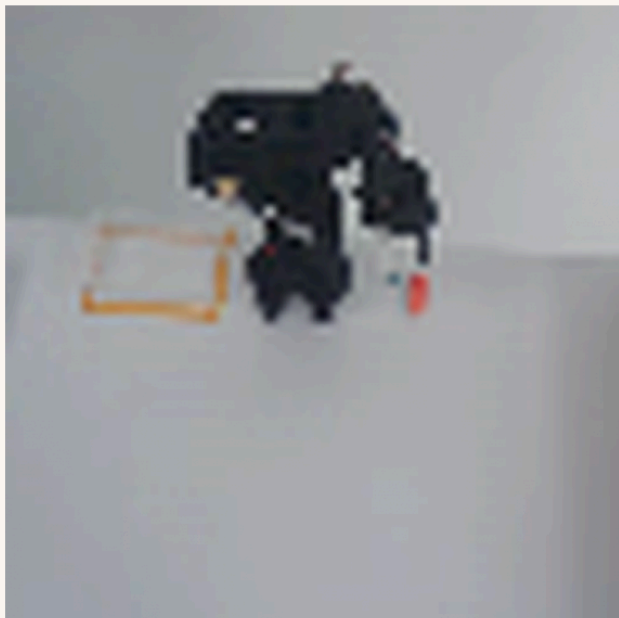
By the end of this part you should be able to say: what a world model tracks at every timestep — not a state, but a *distribution* over states — and exactly why Lecture 3's single vector could not do that job.

STATE · OBSERVATION · UNCERTAINTY

First, look at one row of the data – this is a state, and this is an observation

the **OBSERVATION**

64 x 64 x 3 = 12,288 numbers



ONE row of the dataset: this is what 'state' and 'observation' actually look like

observation.state (6 numbers)

shoulder_pan	-85.15
shoulder_lift	-61.64
elbow_flex	70.42
wrist_flex	73.38
wrist_roll	-89.60
gripper	14.36

action (6 numbers)

shoulder_pan	-85.96
shoulder_lift	-53.18
elbow_flex	62.59
wrist_flex	70.00
wrist_roll	-92.38
gripper	14.37

The state is six numbers. The observation is 12,288.

Lecture 1 taught this abstractly. Here it is concretely, in one row of a real robot dataset:

- **The state**

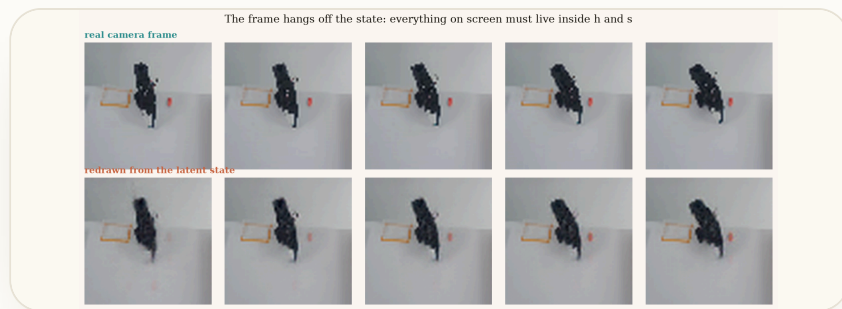
Six joint angles — shoulder pan, shoulder lift, elbow flex, wrist flex, wrist roll, gripper. That is the arm's *entire* configuration.

- **The observation**

A camera image: 12,288 numbers pointed *at* those six numbers — plus a cube, a table, and lighting the robot does not control.

- **And the part in neither**

Is the cube *gripped*, or just touched? Is it about to slip? No joint angle says so, and at the wrong camera angle no pixel does either. That hidden part is what a world model has to infer.



Top: real frames. Bottom: redrawn from the model's internal state — so that state must contain the whole scene.

Now watch what happens when the gripper closes

We just said the state is six joint angles. That is true for the *arm* — but the model has to track the whole situation, and the rest of it is not so obliging. Follow one real moment in this dataset:

1 Before contact — the state is knowable

The camera sees the arm, the camera sees the cube. Six joint angles plus a cube position, all visible. A single vector would be enough.

2 During contact — the cube disappears

The gripper closes over it. For roughly half a second the cube is behind the hand. Its position is no longer in the frame, and no joint angle contains it either. The model has to *hold* it.

3 And "held" is not one number — it is a range

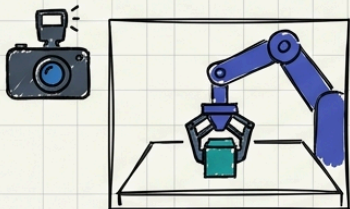
Was the cube nudged as the fingers closed? Did it settle a millimetre left? The model's honest answer is not a position but a *region* — narrow if the grasp looked clean, wider if it did not.

That is the jump this lecture rests on: a world model does not carry a state. **It carries a distribution over states — and the width of that distribution is itself information.** Here is that idea in one picture.

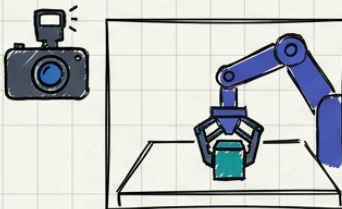
The model does not track a state – it tracks a belief

A BELIEF, NOT A GUESS

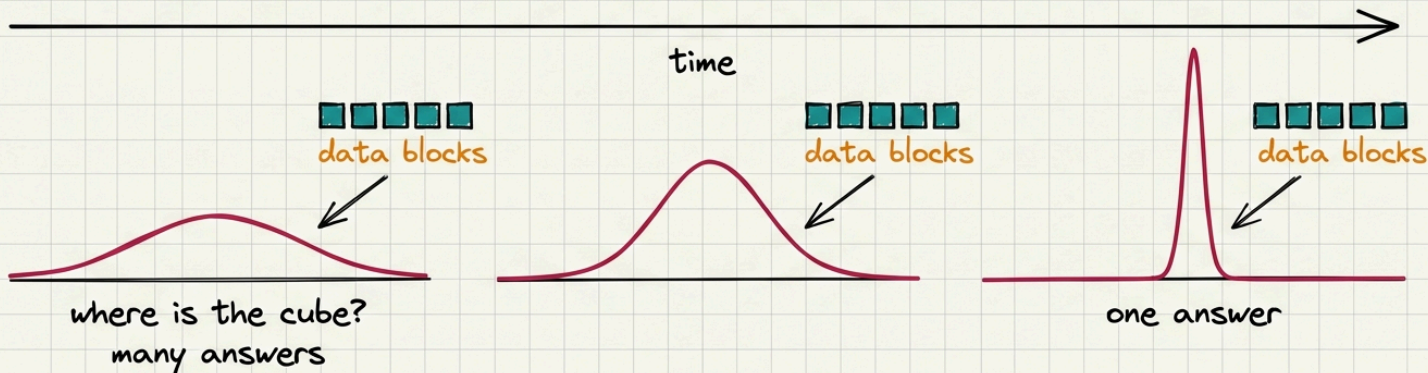
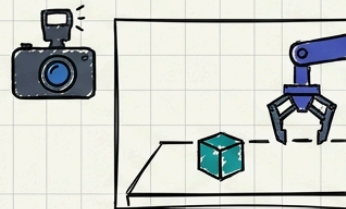
the gripper hides the cube



one step later



the gripper moves away



without evidence the belief widens : one good frame collapses it to a point

When the future forks, an average is a lie

Ask the model: the gripper is closing on the cube — what does the next second look like? There are genuinely two answers: the grasp catches, or the cube slips. A model forced to output *one* state has only bad options:

1

It averages the two futures

Half-gripped, half-slipped — a scene that never occurs in the real world. We watched exactly this in Lecture 3: facing a fork, the model painted a smear instead of a ball.

2

It cannot say "I don't know yet"

A single vector has no way to express *width*. Certain and uncertain look identical, so a downstream planner cannot tell a safe prediction from a wild guess.

3

The fix: carry a mean AND a spread

Track a distribution — a centre and a width per dimension. Wide when the model is guessing, narrow when it knows. You met this machinery in the VAE lecture; here it moves from the encoder to the *transition*.

So what exactly was Lecture 3 missing?

Put the two lectures side by side. Same task — predict the future from pixels and actions — and the gap is now nameable:

● Lecture 3 tracked a point

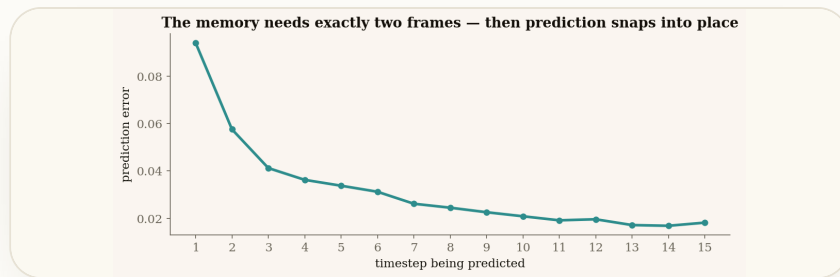
Twelve numbers. One answer, always, with no way to say how sure it was. When the ball's direction was unknowable it still had to commit — so it committed to the average, and we watched the ball smear.

● Today we track a distribution

A centre *and* a width, updated at every timestep. Now "I don't know yet" is something the model can represent instead of something it has to fake.

● And our own measurement said so

Remember Lecture 3's error curve: high at $t=1$, collapsing by $t=3$. We read that as "the memory learned velocity" — true, but incomplete. It was also the belief *narrowing*. We were measuring a distribution all along; we just had no way to represent one.



Lecture 3's error cliff — with today's vocabulary, a belief collapsing as evidence arrives.

Three things Part 1 established

1 The state is small; the observation is huge

Six joint angles versus 12,288 pixels — and you saw both, in one real row of the dataset. A world model's job is to recover the small thing from the huge one.

2 Some of the state is in neither

Is the cube gripped or just touched? No joint angle says it, and during occlusion no pixel does either. That is the part that has to be *inferred and carried*.

3 So the model must track a belief

Wide when it is guessing, narrow when it knows. Everything in the rest of this lecture exists to build and maintain that one object.

Which raises the obvious problem — and it is the whole of Part 2: the dataset contains pixels and joint commands. It does not contain beliefs. Nobody ever wrote down "at frame 40 the model should be 70% sure the cube is held." So where on earth does that belief come from?

Explanation from Rajat

— WHITEBOARD —



PART 2 OF 3

The Machine

By the end of this part you should be able to say: exactly what an RSSM is — the two things it carries at every timestep, why each one is there, and what breaks if you drop either.

DESIGN A · DESIGN B · THE MERGE

Design A: one vector, carried forward – never sampled

Start with the design you already own. This is Lecture 3's model, drawn with today's shapes — and the only change is that we now train it jointly, so the comparison is fair to it.

1 The architecture

Encoder squeezes the frame to 1,024 numbers. A GRU holds **one vector of 288 numbers** and updates it from (that vector, the frame, the action). The decoder paints the next frame from it. That vector is the entire state.

2 What "deterministic" means here

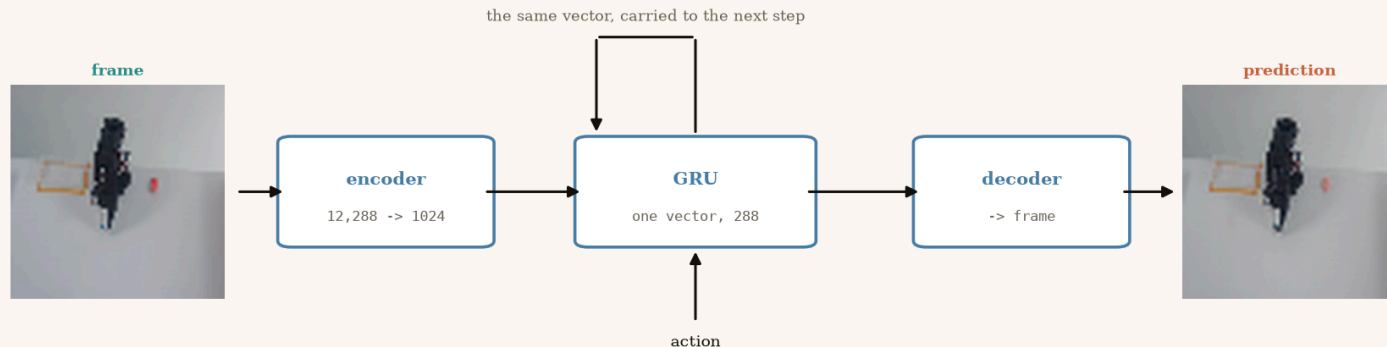
Every arrow is a computation. Nothing is drawn from a distribution. Hand it the same history twice and it produces the identical future twice — no dice anywhere in the machine.

3 What we want to find out

Two questions, and we will answer both with measurements, not adjectives: *what does that vector actually contain?* and *where does this design run out of road?*

Design A, drawn with real shapes – one vector, no dice

Design A — Lecture 3's model, redrawn with today's shapes



Everything is computed. Nothing is sampled. Give it the same history twice and you get the same future twice — which is exactly the strength (facts survive) and exactly the weakness (a fork has to be averaged).

"It carries information" is not good enough. Let us prove what.

People say a recurrent state "carries information forward" and move on. We can do better than that — we can **open the vector and ask it questions**. The technique is called a *probe*, and it takes about ten lines of code:

1

Collect

Run the trained model over 25 real episodes and record, at every timestep, the 256-number state *and* the six true joint angles the robot actually had.

2

Fit the simplest possible readout

A single linear map from the state to the joint angles — no hidden layers, no training tricks. If a plain linear readout can recover a fact, that fact is genuinely *present* in the state, not hidden behind clever processing.

3

And run a control

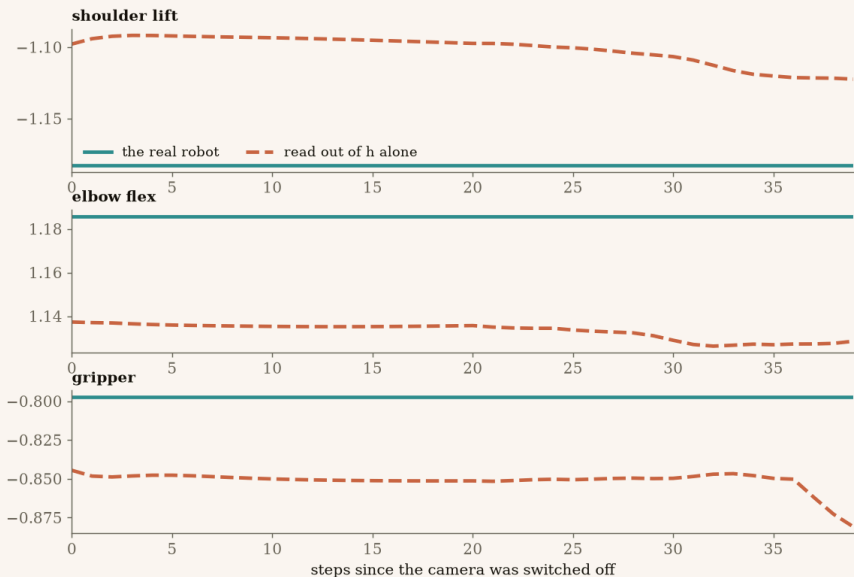
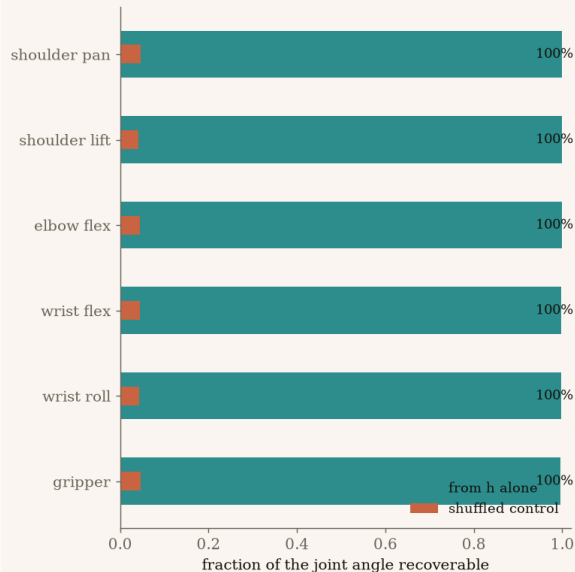
Fit the same readout against *shuffled* labels. If the real fit scores high and the shuffled fit scores near zero, we have found real structure and not an artefact of having 256 free parameters.

The answer: the belt is carrying the arm itself

What is actually riding on the belt? Probe it and see.

The belt is not carrying 'a vector'. It is carrying the arm: a plain linear readout of h recovers every joint angle, and keeps recovering them for 40 steps after the camera is switched off.

a linear readout of h recovers the arm



left: how much of each joint angle a linear readout recovers from the state alone, against a shuffled control · right: those readouts tracking the real robot for 40 steps after the camera is switched off

So this is what "the belt carries the facts" actually means

1

The arm is in there, explicitly

99.9% of every joint angle is recovered by a plain linear readout — against a shuffled control that scores essentially zero. Not "some useful features": the joint angles themselves. The model was never given a joint angle as a target for this vector; it built that representation because predicting frames required it.

2

And it stays there with the camera off

The right-hand panels are the crucial part: those curves are read out of the state alone, 40 steps into a camera-less rollout, and they still track the real robot. Nothing is refreshing them. They are being *carried*.

3

This is the property worth protecting

Whatever we build next must keep it. A design that loses the arm during a one-second occlusion is not usable on a robot — and in a moment we will meet exactly such a design.

But ask it a question with two answers

Freeze the recording at the instant the gripper closes on the cube — the five frames below. Ask the model: what happens next? On a real robot there is more than one honest answer. The grasp catches, or the cube shifts and slips out.

1 Design A must pick one number

Its state is computed, never sampled. Given the same history it produces exactly one future, every single time. It has no vocabulary for "either of these".

2 So the loss makes it split the difference

Under squared error the safest single answer is the *average* of the possible futures — half-gripped, half-slipped. A frame that never occurs in reality.

3 Measured on our robot

Dreamed 60 steps, this design reaches a joint error of **0.409** — forty-five times worse than the design we finish with. Averaging is not caution; it is a confident wrong answer.

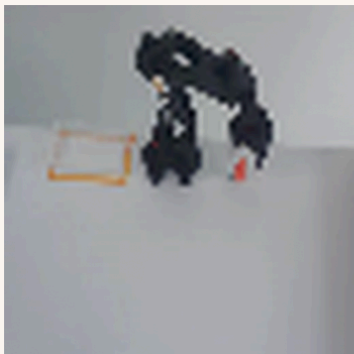
The fork, on the real robot – the moment that breaks Design A

The fork, on the real robot: does this grasp catch, or does the cube slip?

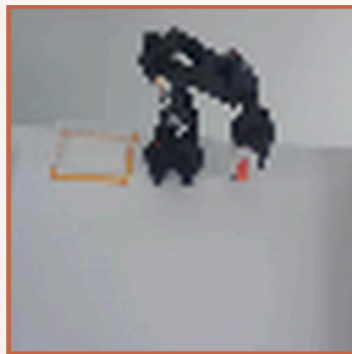
-8 steps



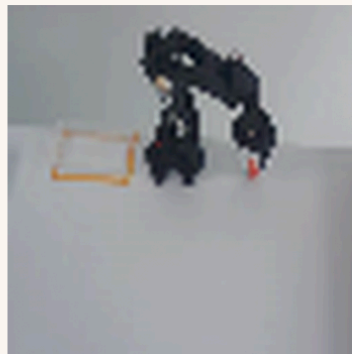
-4 steps



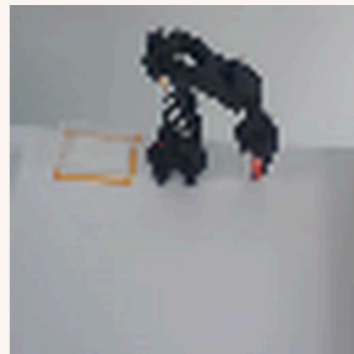
the gripper closes



+4 steps



+8 steps



the highlighted frame is the fork: from here the future genuinely has more than one answer, and Design A can only name one

Design B: sample the state instead – the "dice"

So make the state a **draw from a distribution**. The network outputs a centre and a width; the state is sampled from it. Now the model can answer a two-answer question with two answers – and again, let us not take that on faith. Here is the model, at the grasp moment, asked for six futures:

1 Same state in, different futures out

Six draws from one identical starting state, rolled forward twelve steps and painted. The gripper values differ across the draws – the model is expressing genuine spread instead of one blurred compromise.

2 Each future is sharp, not mushy

This is the whole point. An average of two futures is a blur; a *sample* of one future is a clean, physically plausible frame you could act on.

3 Honest footnote

Our spread is small (about **0.05** in normalised gripper units) – the grasp in this dataset almost always succeeds, so there is little for the model to be unsure about. We amplified the sampling temperature in the figure so you can *see* the mechanism. On a messier task the spread would be the story.

Design B: the dice, made visible – six futures from one identical state

The dice, made visible: six futures the model considers possible

the moment the gripper closes



roll the dice



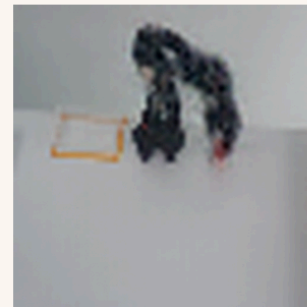
sampled future 1 gripper -0.22



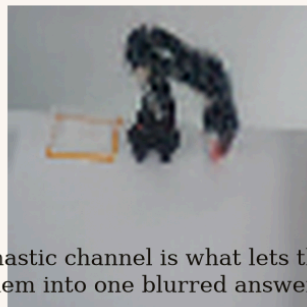
sampled future 2 gripper -0.10



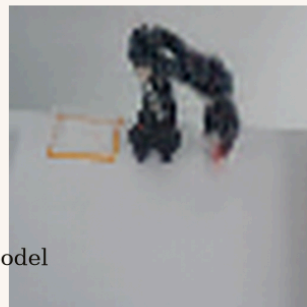
sampled future 3 gripper -0.22



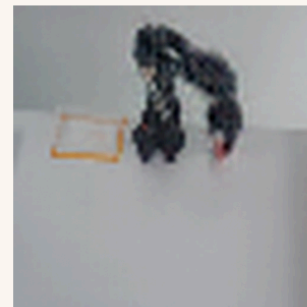
sampled future 4 gripper -0.11



sampled future 5 gripper -0.16



sampled future 6 gripper -0.21



Twelve steps later, six draws from the SAME state. The stochastic channel is what lets the model say 'the future could go several ways' instead of averaging them into one blurred answer.

But now nothing survives the journey

Design B bought honesty at a fork — and lost something Design A had for free. If the state is *redrawn* at every timestep, then every fact must be re-transmitted through a fresh sample, every step, forever.

1 The occlusion becomes impossible

Fifteen frames where the gripper hides the cube is fifteen consecutive dice rolls the cube's position has to survive. It won't. The noise you added for honesty erodes exactly what you were trying to remember.

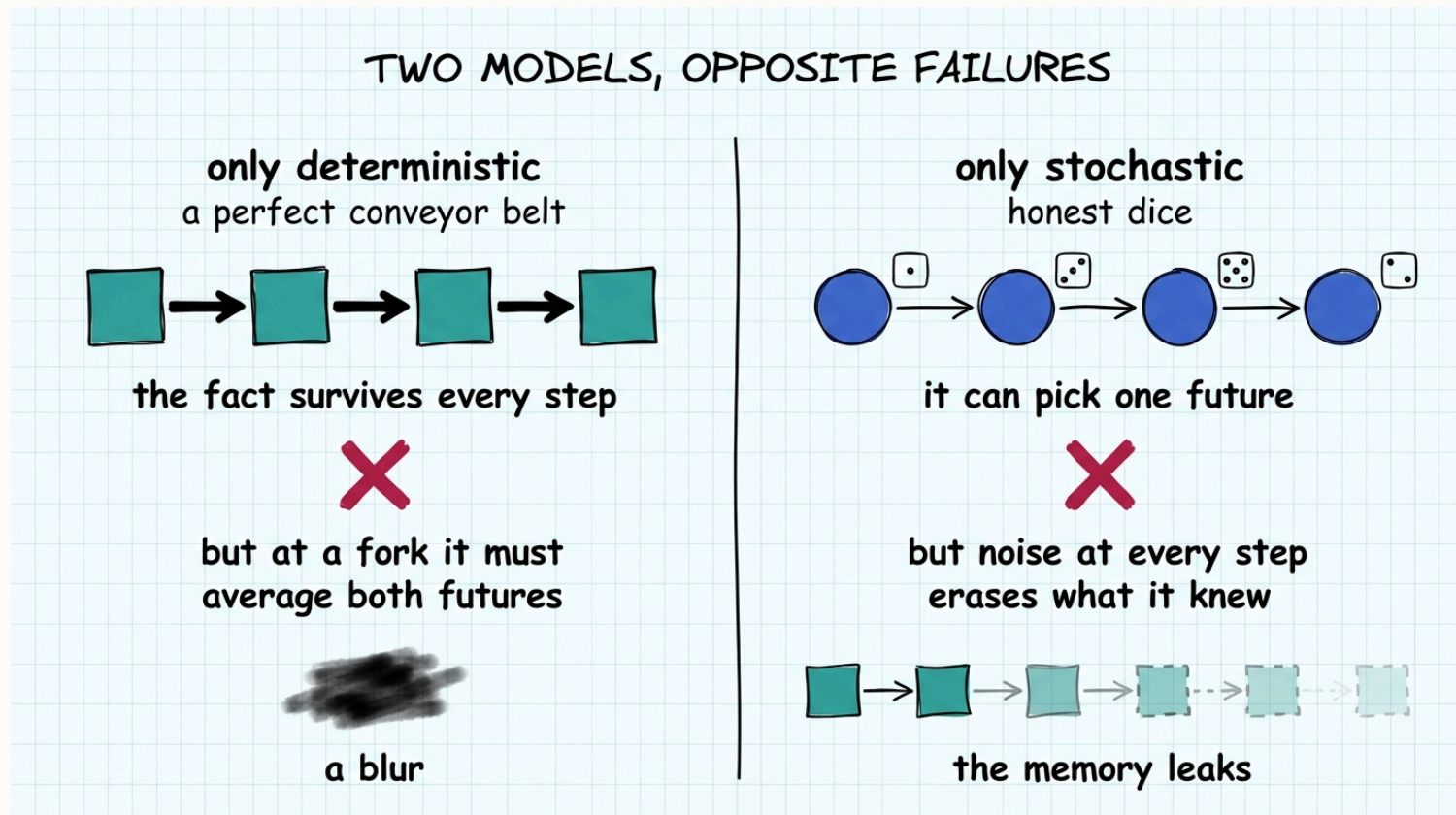
2 We measured it — and it is worse than the argument suggests

Trained on our robot, same data, same budget. Its joint error **rose** during training ($0.26 \rightarrow 0.39$) while its KL cost climbed past 90 — a model paying more and more to re-transmit what it should have kept. Dreamed 60 steps, its joint error reaches **2.60**: the worst of the three by a wide margin, and nearly 300× the design we end up with.

3 So we have two designs and two ceilings

A perfect memory that cannot choose. Honest dice that cannot remember. Neither is a world model you would trust for sixty steps.

Two designs, opposite failures – and neither is enough



left: the fact survives every step, but a fork must be averaged · right: a fork can be chosen, but the memory leaks away

So run both — and let each do the job it is good at

The RSSM's whole idea in one line: **stop choosing**. Carry two things at every timestep, and give each the job the other cannot do.

1 h — the part that is never sampled

256 numbers updated by a GRU, no noise ever applied. This is the vector you watched carry the arm through 40 camera-less steps. It is where facts live.

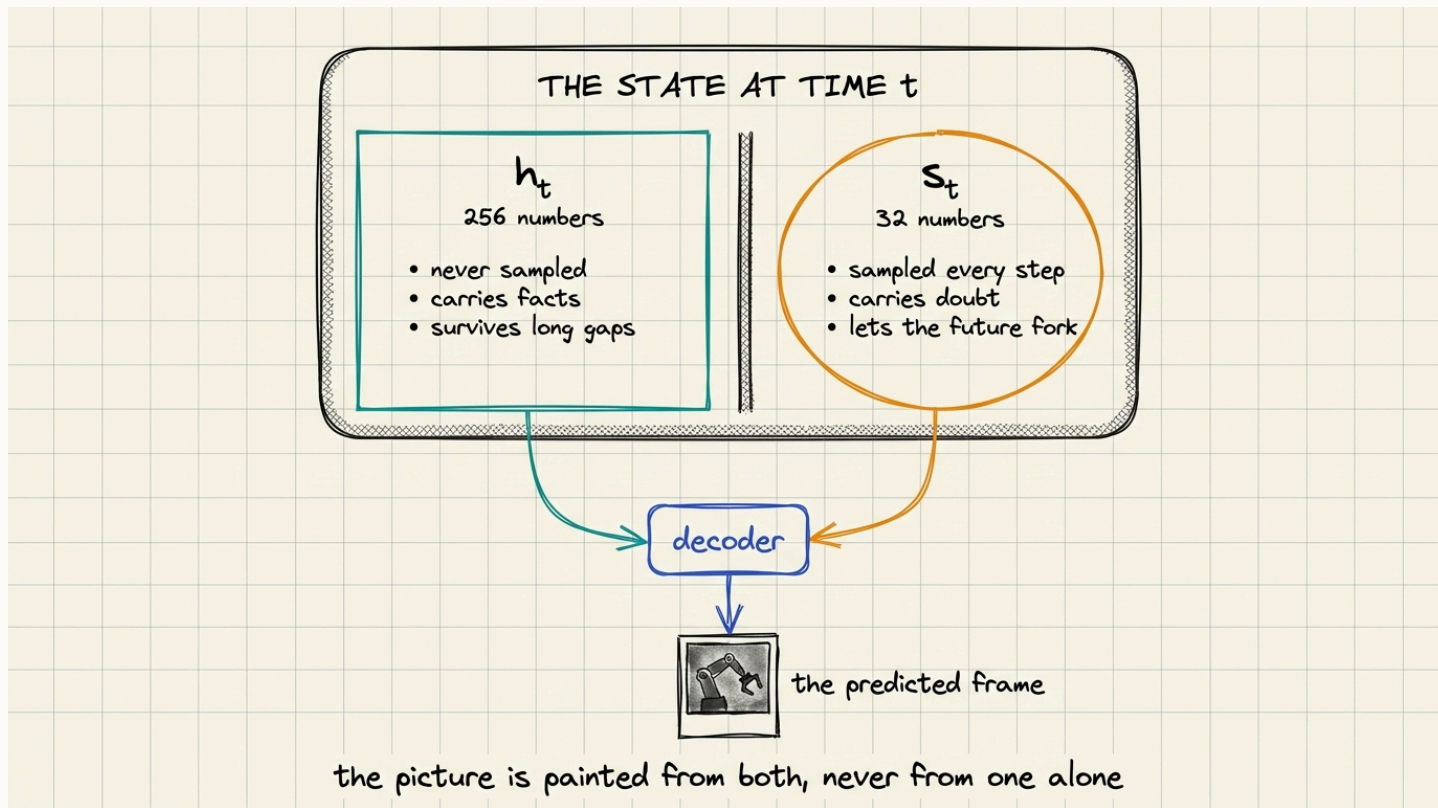
2 s — the part that is always sampled

32 numbers drawn fresh from a distribution the model computes from h. This is where the six futures came from. It is where *doubt* lives.

3 The frame is painted from both together

Not from h, not from s — from the pair. So the picture always reflects both what the model knows for certain and which branch it currently believes it is on.

One state, two parts – this is the whole idea



They are not two models. They are one state with two halves.

This is the sentence to hold on to: at every timestep the model carries one state, and that state has two parts that are read together. Not two pipelines running in parallel — one object, split by job.

1 They are concatenated, literally

In code the decoder receives $[h, s]$ — 256 numbers plus 32 numbers, joined into one 288-number vector. Every prediction the model makes is a function of that combined vector, never of one half alone.

2 They inform each other in both directions

The distribution that **s** is drawn from is computed *from* **h** — the memory decides how uncertain the model should be right now. And the sample **s** is an input to the next **h** — so a doubt, once resolved, becomes a remembered fact.

3 Which is why cutting either one breaks it

Not "degrades it a little" — breaks it. We will show that with the trained model in a moment: freeze **s** and the scene loses its detail; starve **h** of **s** and the model wanders out of its own story.

Where the state lives: Lecture 3 vs today

WHERE DOES THE STATE LIVE?

Lecture 3 - a split state



code z (what it looks like)

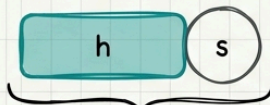
memory h
(what a frame
cannot show)

decoder



the dream must round trip through the code every step

Lecture 4 - one state



decoder



the dream never leaves the state

And this is precisely what changed since Lecture 3

The figure above is the cleanest statement of the difference between the two lectures. Read it twice — top row is what we built last time, bottom row is what we are building now.

1 Lecture 3 split the state in two, and only decoded from half of it

The code **z** held the appearance; the memory **h** held everything a frame could not show. But the decoder only ever read **z**. So to dream one step, the model had to go *out* to a code, decode, and come back — a round trip, every step, each one leaking a little error.

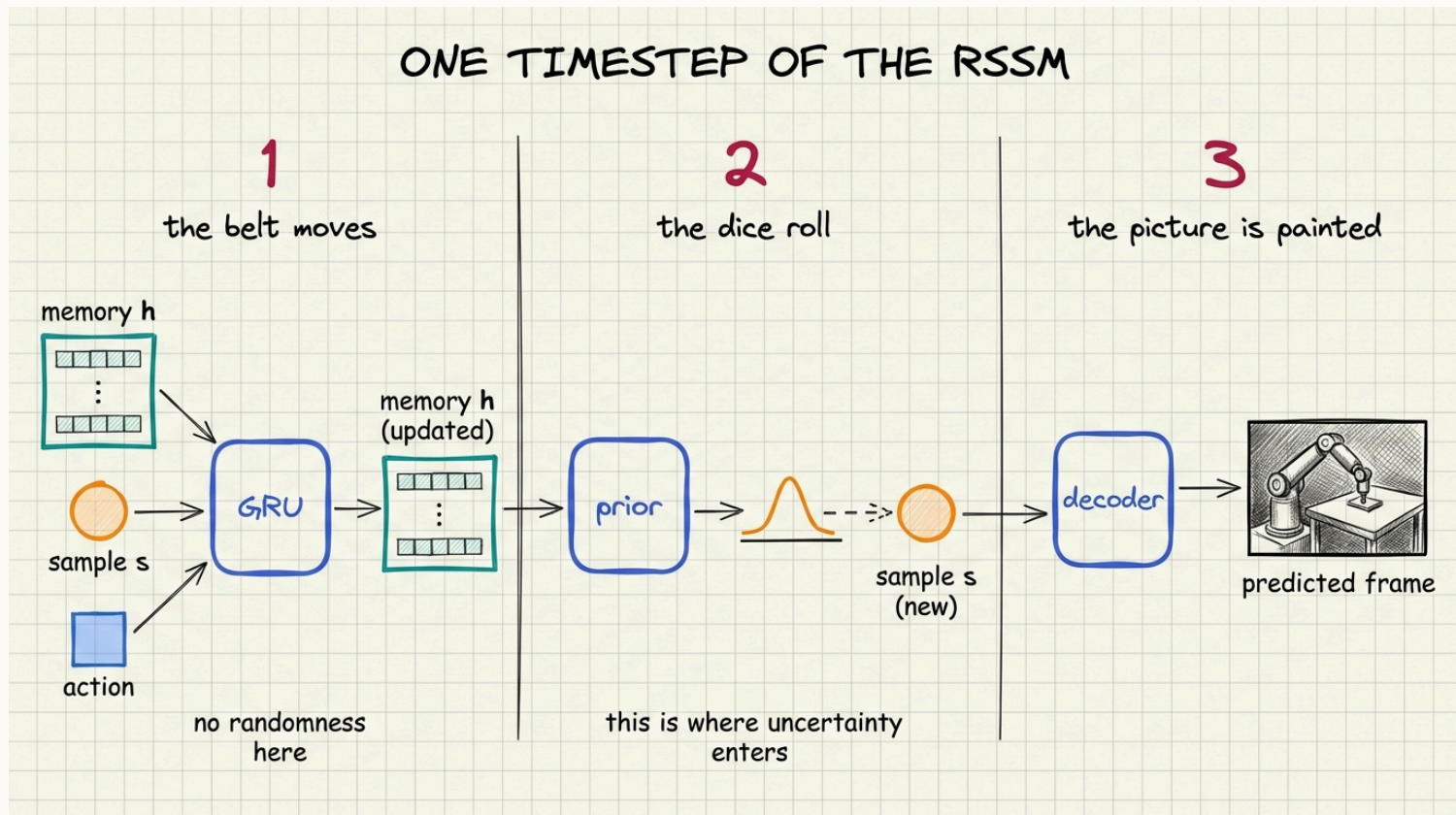
2 Today the state is one object and the frame hangs off all of it

Because the decoder reads $[h, s]$, the state must contain the whole scene by construction. Dreaming is then just $h \rightarrow h \rightarrow h$ internally. We decode a picture only when a human wants to look at one.

3 That is the structural reason the dream lasts

Same data, same task, same horizon — one design leaves its own state space sixty times and one never leaves it at all.

The RSSM: one timestep, three stages



1 · memory h updates from (h, s, action) with no randomness → 2 · a new sample s is drawn from a distribution computed from h → 3 · the frame is painted from h and s together

Walk one timestep, slowly

1

Stage 1 — the memory updates

The GRU takes three inputs: the previous memory **h**, the previous sample **s**, and the **action** that was just taken. It outputs the new memory.

Notice what is absent: no randomness, and no camera frame. This stage alone is Lecture 3's model.

2

Stage 2 — the doubt is drawn

The updated memory is fed to a small network that outputs a *distribution* — a centre and a width. The new **S** is a draw from it. Wide distribution, wildly different draws; narrow one, nearly identical draws. This is the only place randomness enters the whole model.

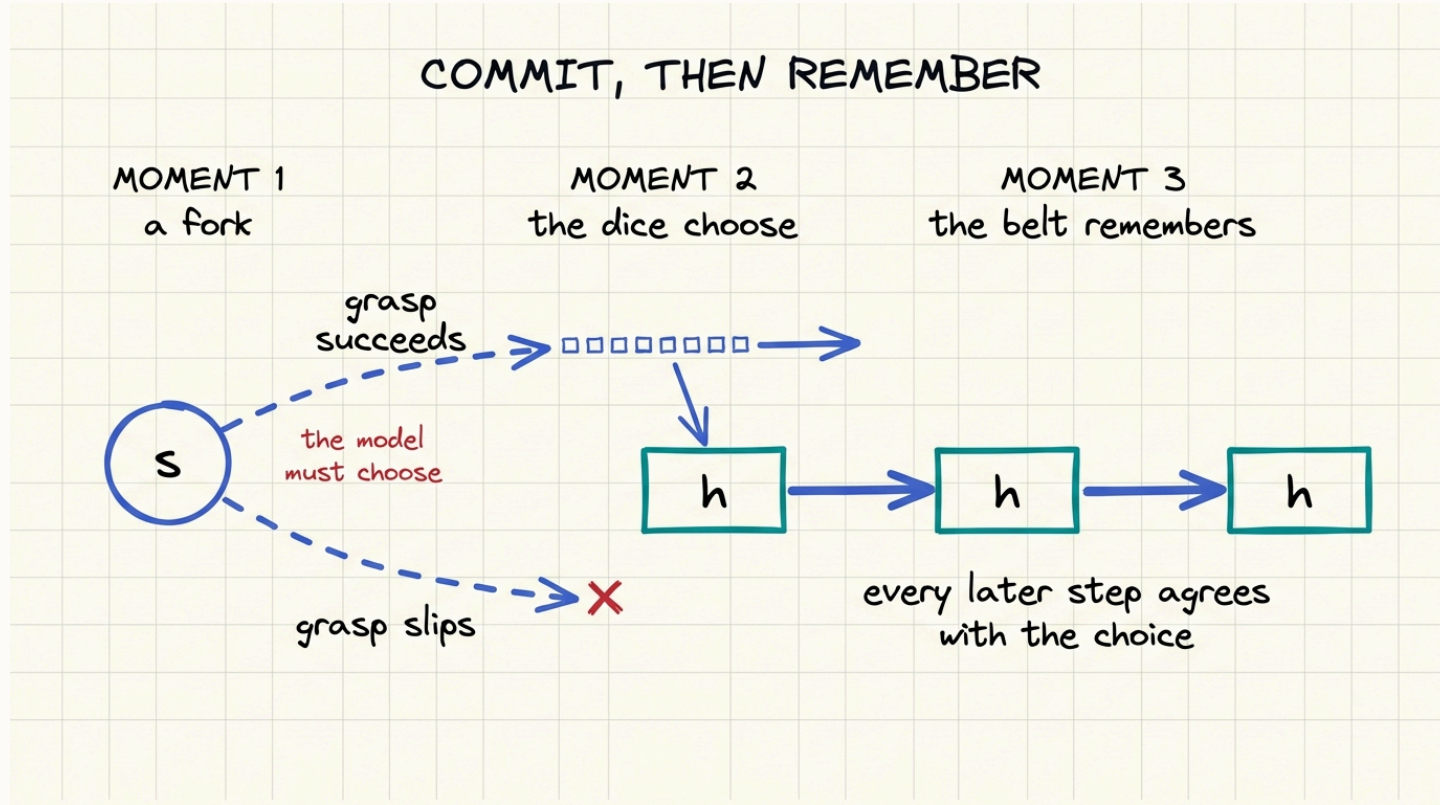
3

Stage 3 — the frame is painted

The decoder reads **h and s together** and paints the predicted image. Then the loop repeats — and here is the crucial detail: that sampled **S** flows straight back into stage 1 of the next step.

That last sentence is the entire trick, and it deserves its own slide.

The mechanism that makes it more than the sum: commit, then remember



Why feeding the sample back matters so much

Suppose at the grasp moment the dice draw "the cube is held".

That sample now flows into the GRU on the very next step — so it stops being a guess and becomes part of the **memory**.

1 The dream stays consistent with itself

Every later step is computed from a memory that contains "the cube is held". The arm lifts and the cube goes with it. The model never contradicts a choice it already made.

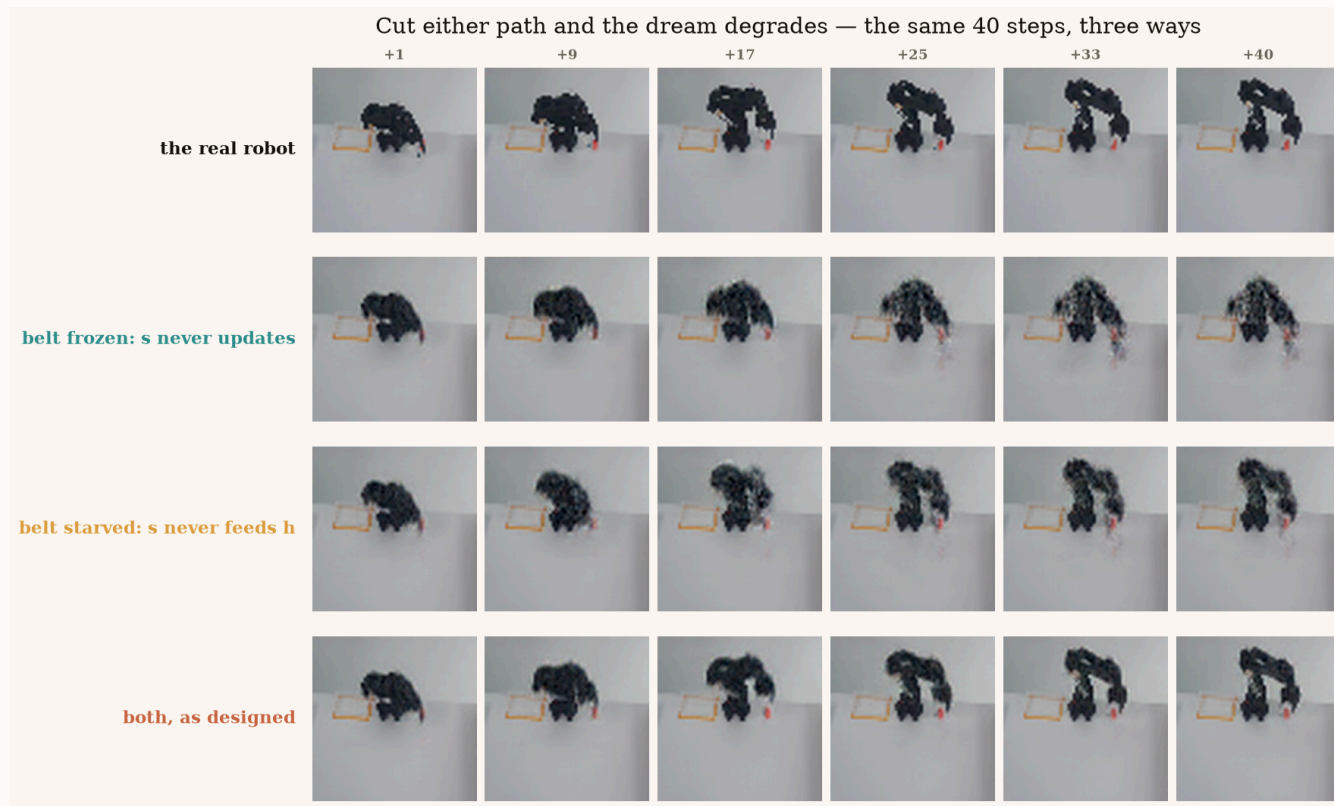
2 Design B could never do this

With no memory to write into, a sampled choice evaporates at the next step — the model can wander out of its own story mid-dream.

3 Design A could never make the choice

It would have averaged "held" and "slipped" into a frame showing neither, and then faithfully carried *that* forward for sixty steps.

The proof: cut either path and the dream degrades



row 2: the sample never updates · row 3: the sample never feeds the memory · row 4: both paths, as designed — the same trained model, three ways of

running it

VIZUARA · BUILD A WORLD MODEL FROM SCRATCH

Why this is a sharper test than three separate models

Notice what that figure is *not*: it is not three different models. It is **one trained RSSM, run three different ways** — so the weights, the data and the episode are identical, and the only thing that changes is which path we allow to do its job.

1 Row 2 — freeze the sample

Keep the memory running, but never let s update. The scene keeps moving, because the belt still carries the arm — but the model can no longer revise which branch of the future it is on, and the detail decays.

2 Row 3 — starve the belt

Keep sampling s every step, but stop feeding it into the memory. Now the model chooses at every fork and immediately forgets what it chose. It cannot stay inside its own story.

3 Row 4 — both, as designed

Identical weights, identical actions — and the dream stays with the robot. The two paths are not two features bolted together; they are **load-bearing for each other**.

Why this makes dreams last

1 The dream never leaves the state

Lecture 3's dream round-tripped through the code space at every step — encode, predict, decode, feed back — and every trip leaked. Here the belt runs internally: state → state → state. We decode a picture only when we want to *watch*.

2 The only noise is deliberate

Randomness enters through one channel built for it, at the moments the world genuinely forks — not smeared across every step of the memory.

3 Everything was trained together

Encoder, transition and decoder share one state and one loss. The representation is shaped *by* the prediction task from the first gradient step — the thing our separately-trained Lecture 3 model could never get.

Explanation from Rajat

— WHITEBOARD —



PART 3 OF 3

The Training

By the end of this part you should be able to say: how a machine like this is taught — where the training signal comes from when nobody can tell you the right answer, and why it takes exactly two loss terms.

THE MISSING LABELS • LOSS 1 • LOSS 2

We have a machine. But what do we train it against?

We now know what the model carries: a memory **h** and a sample **s**. To train it we need a target for those numbers — and here is the difficulty: **the dataset does not contain them.**

1 What the dataset has

Frames and joint commands. That is all. Nowhere in those 11,939 rows is there a column saying "at this moment the state should be these 288 numbers".

2 And nobody could write one

The state is the model's own private summary — its size and meaning are ours to choose. There is no ground truth to compare against, and for the part that matters most — *is the cube held?* — no human labelled it either.

3 So we need targets that we DO have

Two of them, and this is the whole of training: **the frames we recorded**, and **the model's own second opinion**. One becomes loss 1, the other becomes loss 2.

Loss 1 – repaint the frame, and see if it matches

The first target is the one thing we definitely recorded: the **frame itself**. So we bolt a decoder onto the state and demand it repaint that frame.

● What it does

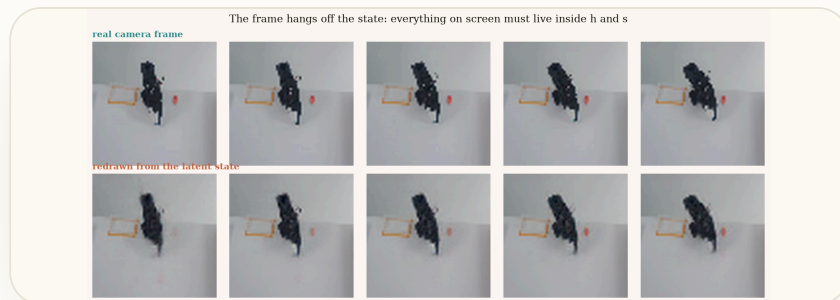
Take the state, paint a 64×64 image, compare it pixel by pixel with what the camera actually saw. The error is loss 1.

● Why we need it — it forces the state to CONTAIN the world

Without this term the state could be anything at all, including nothing. The only way to repaint the arm, the cube and the table is to have kept them. This is the term that fills the state with content.

● And why it is not enough on its own

Lecture 3 already had this term and still failed. A state can be perfectly repaintable and still be impossible to *predict forward* — that is exactly the disease we diagnosed last time. Loss 1 fills the state. It does nothing to make it predictable.



Loss 1 at work: top real, bottom repainted from the state, on held-out episodes.

Loss 2 – the trick: ask the same question twice

We need a term that rewards the state for being *predictable*.

There is no label for that either — so we manufacture one, by asking the model to compute the same quantity twice, under different rules.

1 Ask once WITH the frame

"Given the memory, the action, *and the camera image you just received* — what is the state now?" That is the **posterior**. It has all the evidence, so its answer is the better one.

2 Ask again WITHOUT it

"Given only the memory and the action — what is the state now?" That is the **prior**. It must guess blind. This is exactly the situation the model will face when it dreams.

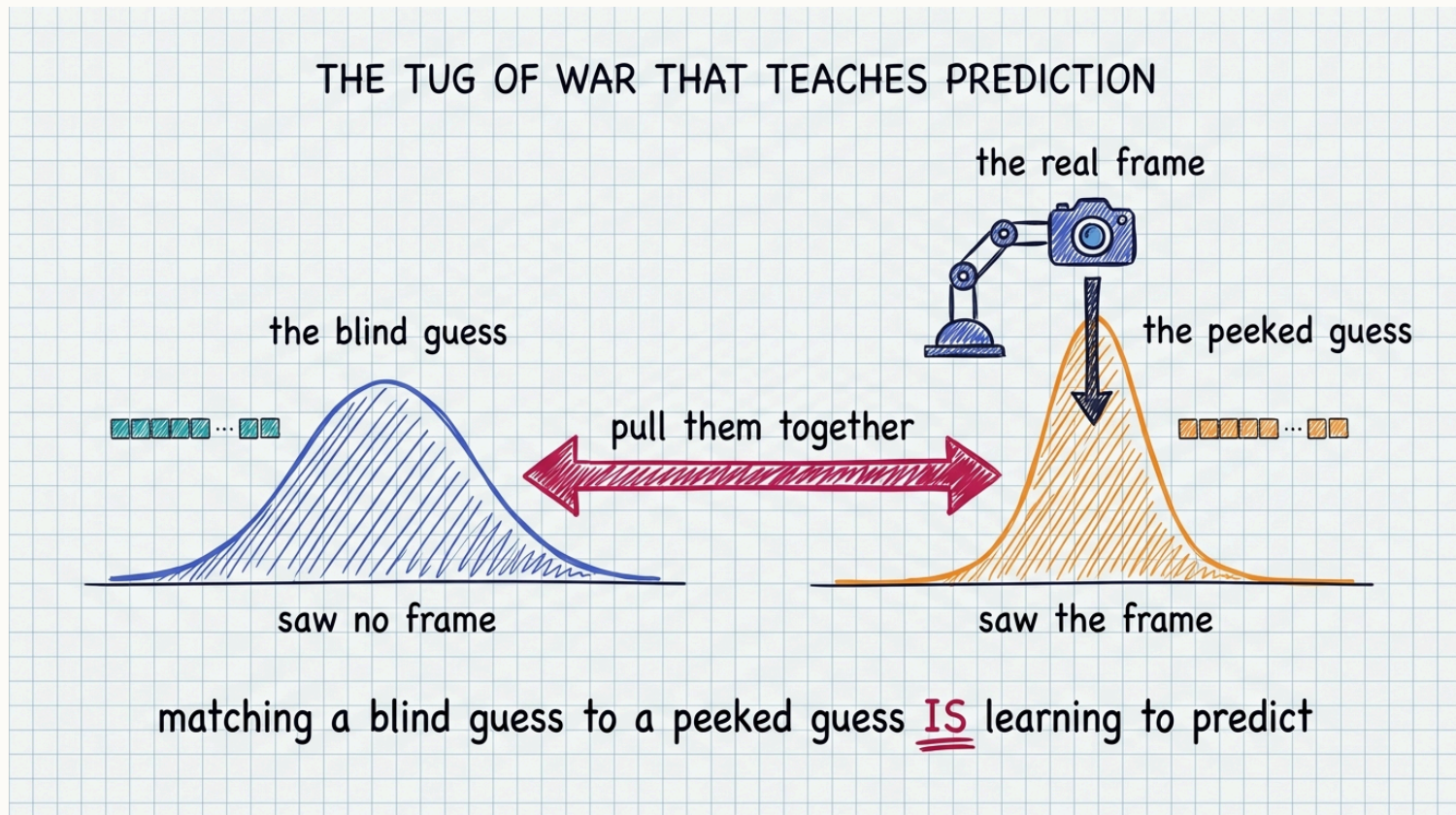
3 Loss 2 is the distance between those two answers

Minimise it, and you are training the blind guess to match the informed one. **There is no way to win that game except to actually carry forward whatever determines the next frame** — which is the definition of predicting. The model grades itself, so no labels are needed.

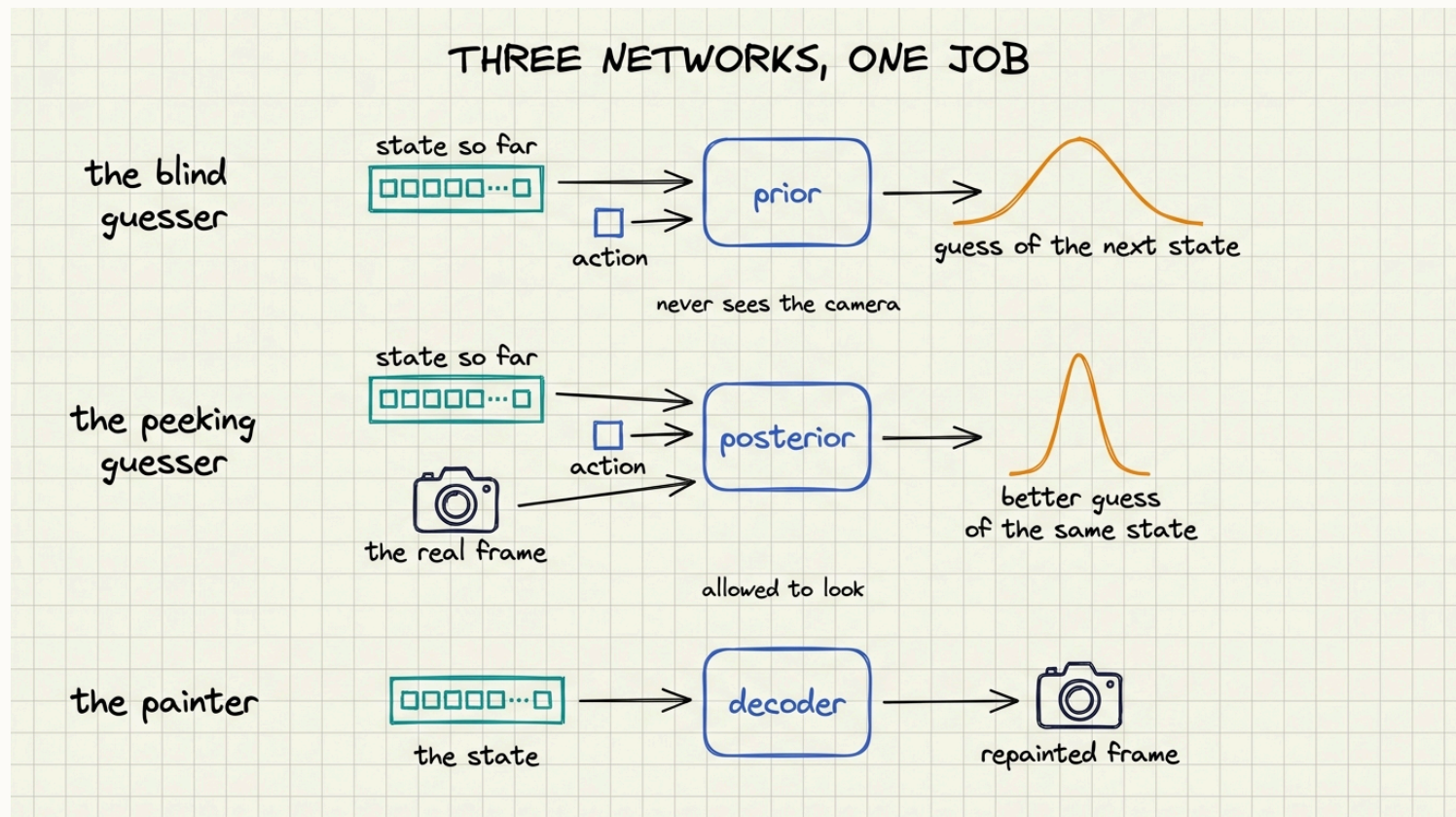
The two losses do different jobs, and you need both. Loss 1 puts the world *into* the state. Loss 2 makes the part that is in there *forecastable*.

Lecture 3 had only the first — which is precisely why its dream fell apart.

The loss that turns three networks into a world model



The three networks – and what each one is allowed to see



the prior never sees the camera · the posterior does · the decoder proves the state holds the whole scene

And loss 2 pulls in both directions — which fixes Lecture 3

One subtlety worth a slide, because it is where the real magic is.

Loss 2 measures a gap between two networks — so the optimiser can close that gap by moving *either* of them.

1 It improves the predictor

The prior is pushed to guess better — to carry the ball, the cube, the velocity. This is the direction everyone expects.

2 But it also disciplines perception

The posterior is *punished* for encoding anything the prior could never anticipate. Perception is pushed toward representations that are **predictable in the first place**.

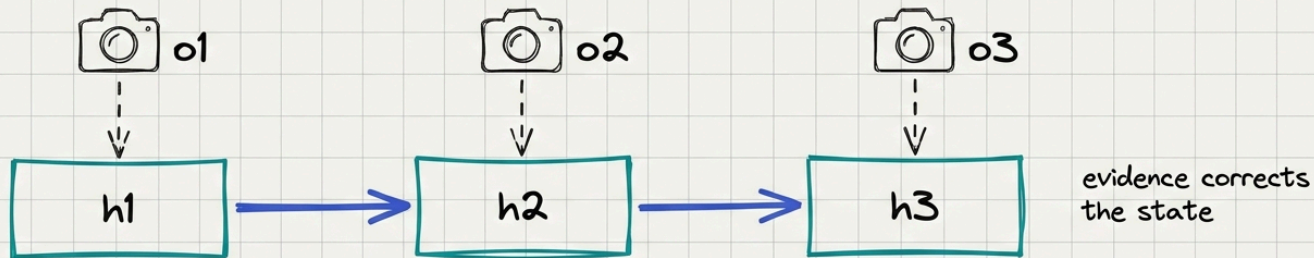
3 Which is the fix we hand-rolled last time

In Lecture 3 we bolted on a "smoothness" penalty forcing consecutive frames to nearby codes — a blind guess at this principle. Here it is not a patch. It falls out of asking one question twice.

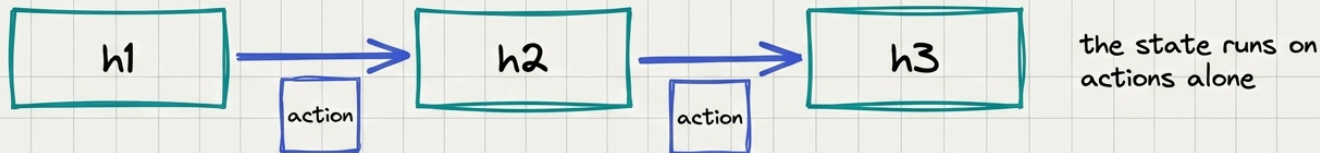
Same model, two modes – and the arrows tell you which

TWO MODES, ONE MODEL

TRAINING - the frames are there



DREAMING - no frames at all



Now watch both losses fire on one real timestep

We have the machine and we have the two losses. Let us put them together on **one actual timestep of one actual episode** — held-out episode, timestep 60 — and print what comes out of every arrow.

1 Carried in from timestep 59

The memory **h**, the sample **s**, and the six joint commands that were actually sent to the robot at step 59.

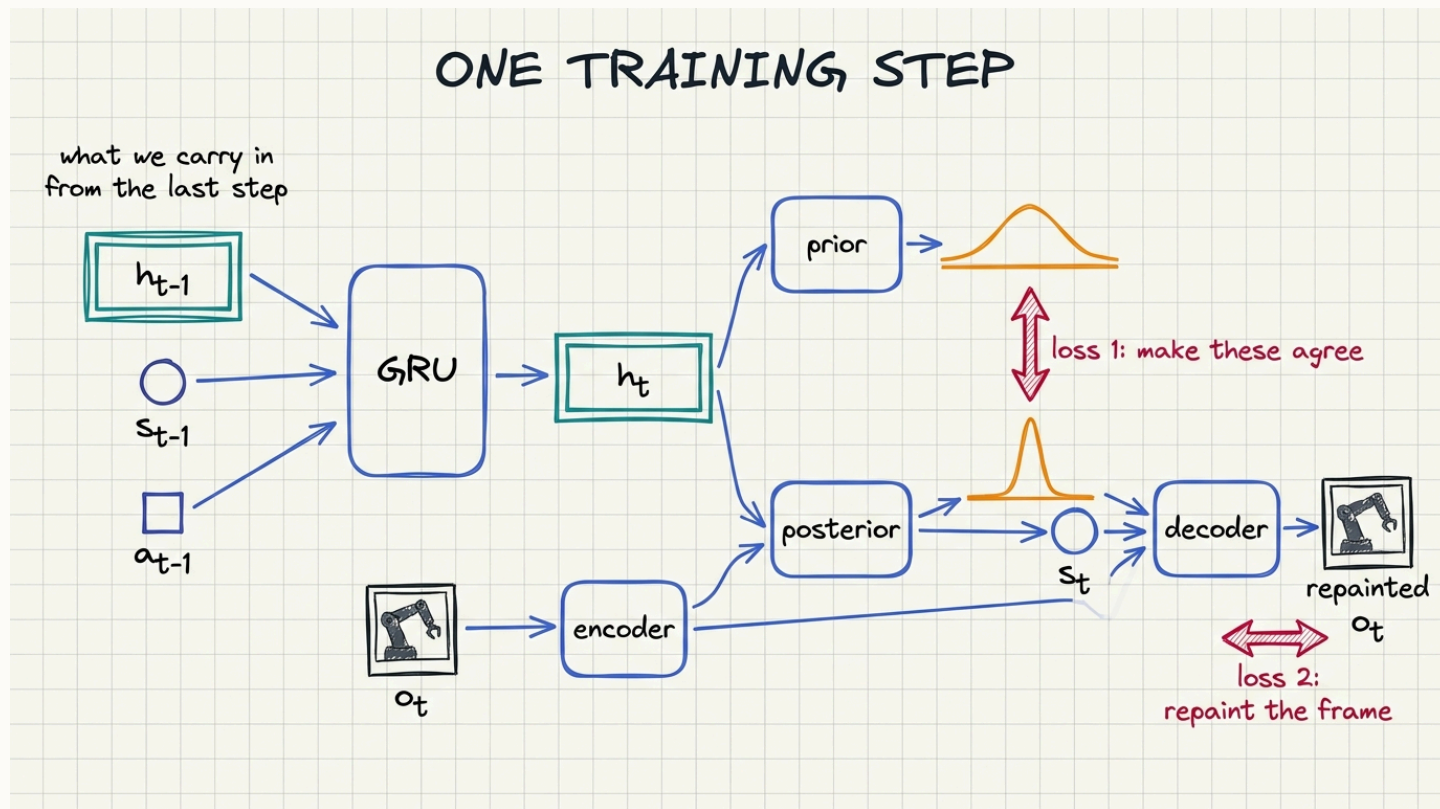
2 Both guessers answer

The prior sees only the updated memory. The posterior also sees the encoded camera frame. Two answers to the same question — and the gap between them is loss 2.

3 The decoder repaints

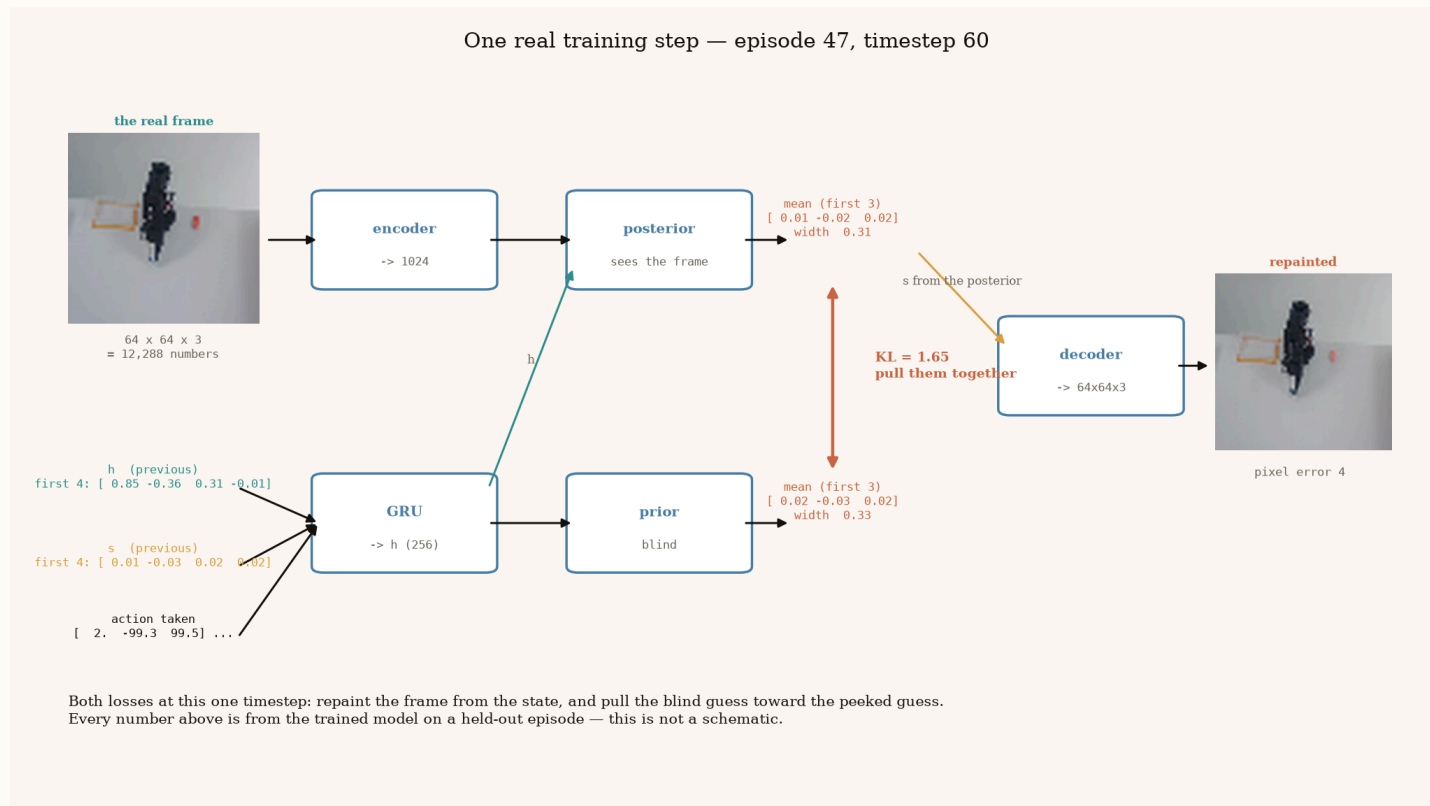
From $[h, s]$ together, and the pixel error against the real frame is loss 1. Add the two, take one gradient step, move to timestep 61. That is training.

One training step, drawn with its timesteps



note the encoder arrow: it reaches the posterior and nothing else — that asymmetry is what makes loss 2 a prediction lesson rather than a copying exercise

The same step, with the real numbers from the trained model



held-out episode, timestep 60 ; every value above is read out of the trained model — this is not a schematic

Three things to notice – and one comparison to make

1 The frame enters on exactly one path

Follow the arrow out of the encoder: it reaches the posterior and stops. The prior never touches it. Break that rule and loss 2 becomes trivial – both guessers would just copy the frame and learn nothing about time.

2 In a trained model the two guesses nearly agree

That near-agreement *is* what "this model can predict" looks like numerically. Early in training the gap is enormous; watching it close is watching the model learn physics.

3 Both losses are computed at every timestep

Not once per episode – 32 times per training sequence, on every sequence, for 16,000 steps. The model is asked "could you have predicted this?" millions of times.

And here is the comparison with Lecture 3. Our old model computed a state from the frame and then, *separately and afterwards*, trained a predictor on top of frozen codes. It was never once asked "could this state have been predicted without looking?" – because it had no second opinion to compare against. Today's model asks that question at every timestep, and the answer becomes a gradient. Same data, same robot; one model is graded on prediction from the first step, the other never is.

Three things Part 3 established

1

There are no state labels, and none are needed

The model supervises itself with two things it already has: the recorded frames, and its own second opinion.

2

Loss 1 fills the state • loss 2 makes it predictable

Reconstruction puts the world into the state. The prior-versus-posterior gap forces what is in there to be forecastable. Drop either and you have Lecture 3 again — or a state that contains nothing.

3

Everything trains together, from the first gradient

Encoder, memory, prior, posterior and decoder share one state and one loss. No stage is frozen and handed to the next.

The machine is built and the training is defined. All that is left is the honest question: **does it work?**

Explanation from Rajat

— WHITEBOARD —

The test: five held-out episodes, sixty steps, no camera

The experiment: take five pick-and-place episodes the model has *never* seen. Give it five frames to warm up. Then switch the camera off and hand it only the joint commands that were actually sent — and ask it to imagine the next sixty steps: two full seconds of robot motion.

1 The model

7.7 million parameters — a convolutional encoder, a 256-number deterministic memory, a 32-number stochastic state, a decoder, and a small head that reads the six joint angles back out of the state.

2 The training

45 tele-operated episodes. Roughly 50 minutes on a single GPU. One loss: repaint the frame, read out the joints, and pull the blind guess toward the peeked one.

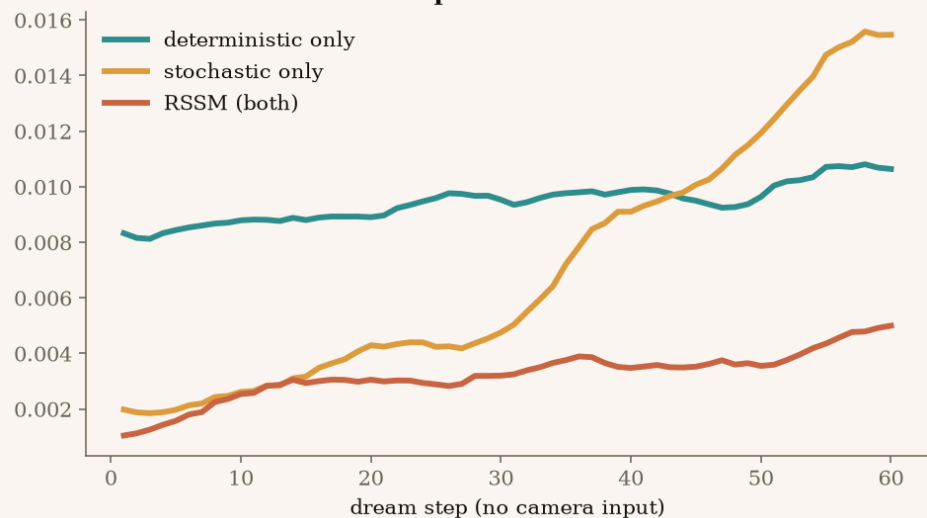
3 What would failure look like?

Exactly Lecture 3: error climbing step after step, the scene dissolving into a soft average. That is the honest thing to watch for.

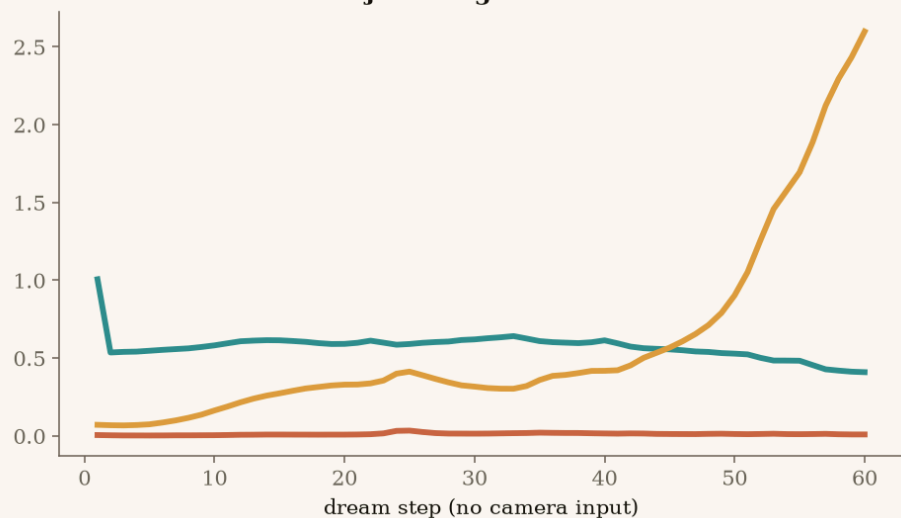
The ablation: same data, same budget, only the state design changes

Same data, same budget — only the state design differs

pixel error



joint-angle error



three models : identical data, identical training budget, identical seed · matched parameter counts (~7.7M each)

VIZUARA · BUILD A WORLD MODEL FROM SCRATCH

We built all three. The argument holds.

Everything in Part 3 was an argument until now. So we trained all three designs ourselves — same 45 episodes, same 16,000 steps, same seed, parameter counts matched — and ran the identical 60-step test:

DESIGN	PARAMETERS	PIXEL ERROR @60	JOINT ERROR @60
deterministic only	7,718,313	0.0106	0.409
stochastic only	7,543,881	0.0155	2.598
both — the RSSM	7,665,993	0.0050	0.009

1 Each design fails in the way we predicted

Deterministic-only holds the scene but cannot represent a fork: **0.409**. Stochastic-only can fork but cannot carry anything: **2.598**, the worst of the three — its state genuinely leaks away.

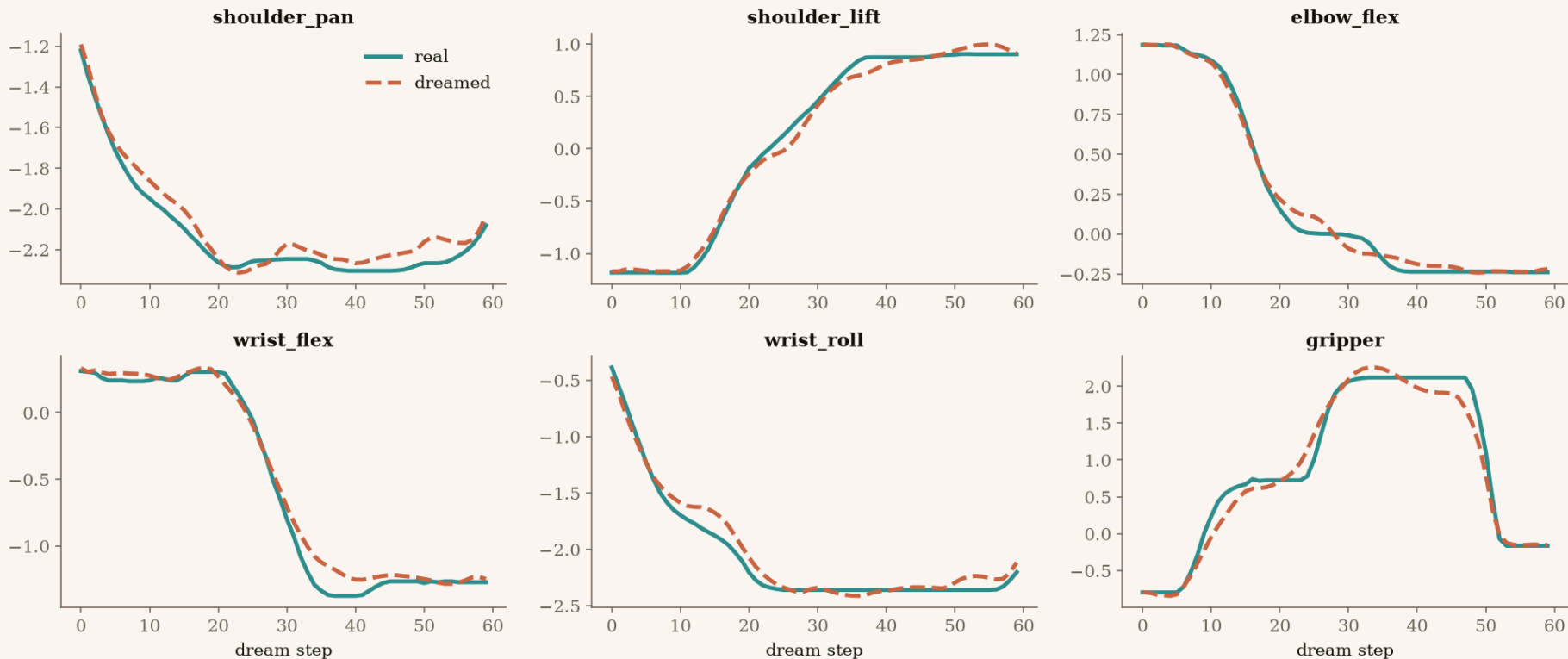
2 The combination is not a small improvement

0.009 — 45× better than deterministic-only and 288× better than stochastic-only, at matched parameter counts. This is not tuning; it is the state design.

And note the pixel column. It separates the three far less than joint error does — pixels are dominated by the static table and background, while the *robot* is where the designs actually differ. Choose the metric that measures the thing you care about.

Sixty steps of pure imagination, against the real robot

60 steps of pure imagination — the dreamed joint angles track the real robot



solid = the real robot · dashed = dreamed with no camera input · note the gripper closing on cue

The model knows when the gripper closes

Look at the sixth panel. The gripper signal drops sharply as the hand closes on the cube — and the dreamed curve drops at the same moment, without ever seeing a frame of it.

All six joints track

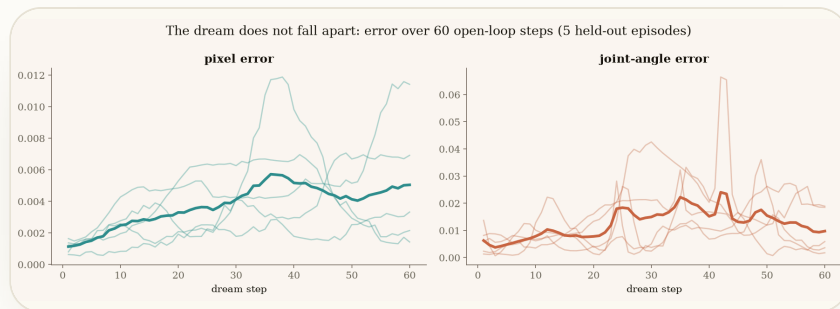
Not just the smooth ones. The sharp, discrete event — contact — is predicted at the right timestep.

The error stays flat

Joint error runs about 0.002–0.02 from step 1 through step 60. It does not climb. On one episode the error at step 60 is *lower* than at step 30 — the dream re-converges toward reality instead of running away.

This is the whole promise

A model you can run forward, on its own, for a horizon long enough to plan with. Lecture 3 could not do this. The state design is the difference.



Error vs dream length, five held-out episodes. Flat, not climbing.

What we measured, and what we did not

One claim we expected to make — and could not:

1 What we did see

At the very first timestep, with no history at all, the model's uncertainty is at its maximum — and **one frame collapses it**. The belief-narrowing story of Part 1, measured on a real robot.

2 What we did not see

We expected uncertainty to *spike* at contact — the grasp is the moment the future genuinely forks. Averaged over the state's dimensions, it does not. Either the forks live in a few dimensions the average washes out, or 50 episodes of a reliable grasp simply contain little for the model to be unsure about.

3 Why we are telling you

Because the alternative is to show you a plot that flatters the story. A result you cannot reproduce is worth nothing — and "the experiment did not confirm this" is itself a finding.

Four things to keep

1 A world model tracks a belief, not a state

A centre *and* a width. Certainty is information; a model that cannot express doubt will invent confident nonsense.

2 Prediction is taught by a tug-of-war

Make the guess-without-looking match the guess-with-looking. That single term is what turns an autoencoder into a world model.

3 Keep the two paths separate

A deterministic belt for facts you are sure of; sampled dice for the moments the world forks. Mix them into one channel and you lose both.

4 Train the pieces together

A representation learned without the prediction task will not survive the prediction task.

Next: acting inside the dream

You now have a model that can imagine two seconds of a real robot's future from actions alone. The next question is the one this whole series has been walking toward: **what if the robot practised in there?**

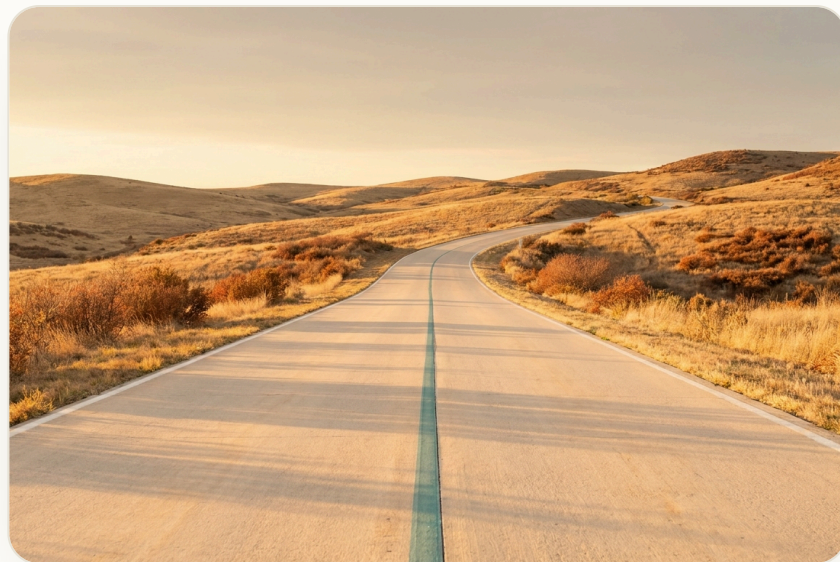
● Dreamer

Freeze this world model, put an agent inside it, and train the agent entirely on imagined rollouts — thousands of them, at no cost, with no robot at risk.

● And the honest question

Does skill learned in a dream transfer to the real world? We will test it, and report what we find.

●●● Vizura AI · Build a World Model from Scratch



First we learned the world. Then we made the dream last. Next, we act inside it.